# ZADANIE 1.1

## *Zastosowane algorytmy*

Zadanie polega na znalezieniu punktów, które są bliżej niż podane na wejściu m, a następnie ich scalenie. Moim początkowym pomysłem było rozwiązanie zachłanne, czyli bierzemy punkty, porównujemy czy są wystarczająco blisko, by je scalić, a jeśli tak, to od razu tworzymy nowy punkt, a stare dwa dezaktywujemy. Działa to poprawnie dla danych z treści, jednak problem pojawia się, gdy 3 (lub więcej) punktów jest umieszczone na jednej prostej (niekoniecznie dokładnie na prostej, ale generalnie są rozciągnięte; punkty oddalone skrajne są od siebie oddalone o więcej niż m). Takie dane to np.

A = (1, 1)

B = (1, 2)

C = (1, 3)

m = 1

Algorytm zachłanny najpierw scali jedną parę, czyli powiedzmy A z B, powstanie

C = (1,3)

D = (1, 1.5)

A wtedy punktu C nie da się scalić z punktem D, czyli ewidentnie nie pasuje.
Dla tego przypadku (gdzie B jest środkiem odcinka AC) okazuje się, że punkt B jest tym faktycznie istniejącym pachołkiem, a punkty A i C są zduplikowanymi odczytami. Z względu na takie przypadki nie można punktów scalać w locie, podejście zachłanne nie działa.

Zamiast tego możemy spojrzeć na punkty jako na strukturę zbiorów rozłącznych. Gdy dwa punkty są bliżej niż m, to dodajemy je do wspólnego zbioru, ponieważ muszą być odczytami tego samego pachołka. Idealna do tego jest struktura Find-Union, gdzie operacja Union jest wykonywana gdy dystans między A i B jest mniejszy niż m. Gdy sprawdzimy wszystkie punkty, to obliczamy (dla zbiorów o liczbie elementów większej niż 1) nowy punkt uśredniony.

By zoptymalizować szukanie punktów, gdy mogą być bliżej niż m dzielę cały układ punktów na siatkę. Samo m to niejako promień scalania, więc sprawdzając 8 komórek siatki wokół komórki z punktem (oraz samą komórkę w środku) mam gwarancję, że żaden punkt bliższy niż m nie został pominięty.

Aspekt techniczny numerowania: dla każdego punktu sprawdzamy komórki w każdym kierunku, wobec tego jest ryzyko, że sprawdzając punkt znajdujący się w komórce na granicy siatki (np. komórkę na lewo od skrajne lewego punktu) wyjdziemy poza zakres, co skutkuje błędem. Dlatego siatka jest ma “ramkę”, w każdą stronę ma dodatkowe komórki jak na obrazku:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 |
| 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 |
| 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 |
| 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 |
| 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
| 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

Zielone pola to te używane, w których mogą znajdować się punkty, natomiast czerwone to zabezpieczenie i na pewno są puste, natomiast pozwala to ograniczyć konieczność każdorazowego sprawdzania, czy nie jesteśmy na granicy siatki. Z tego powodu pierwsza komórka użytkowa ma id równe liczbie prawdziwych kolumn + 2 + 2. Przeskok do góry to dodanie liczby prawdziwych kolumn + 2, a przeskok na prawo to +1.

### *Dokładny opis działania programu*

**struct pacholek** - każdy pachołek ma swoje id, współrzędne, zmienna logiczną przechowując czy jest aktywny (jeśli nie, to został scalony), oraz id komórki, w której się znajduje.

**vector <pacholek> P** - vector zawiera wszystkie pachołki

**vector <pacholek> hull** - przechowuje pachołki, które należą do otoczki

**unordered_map <int, vector <int>> grid** - mapa, w której klucz to id komórki, a vector przypisany do klucza to id wszystkich pachołków jakie się w niej znajdują.

**int parent[1000]** - element Find-Union, przechowuje zbiór, do którego należy dany punkt

**int size[1000]** - element Find-Union, przechowuje wielkość zbioru (ile punktów do niego należy)

**vector <vector <pacholek>> sets(1000)** - tutaj przechowywane są pachołki należące do danego zbioru

Powyższe wielkości tablic i sets trzeba dobrać do charakterystyki konkretnej sytuacji, tutaj takie wystarczają.

**bool cmp** - custom comparator, sortuje pachołki po x

**bool cmp2** - custom comparator, sortuje pachołki po id

**bool is_closer** - oblicza kwadrat odległości między P1 i P2, porównuje wynik do m^2. Nierówności zachowują swój stosunek przy podniesieniu do kwadratu, także pierwiastkowanie nie jest potrzebne.

**int calculate_cell** - oblicza odległość P1 od początku siatki, wyznacza w którym wierszu i kolumnie się znajduje, a następnie oblicza dane id komórki. “columns + 2” to id pierwszej użytkowej komórki, dlatego też jest dodawane.

**int find_set** - element Find-Union, znajdujemy zbiór do którego należy pachołek o danym id

**void unite** - “dopinamy” mniejszy zbiór do większego, więc oba pachołki po scaleniu należą do tego samego zbioru, a wielkość zbioru większego została jeszcze zwiększona o wielkość zbioru mniejszego.

**double cross_product** - oblicza iloczyn wektorowy między 3 pachołkami, a to pozwala (przy określaniu otoczki) na stwierdzenie, czy dany punkt powoduje skręt w prawo, w lewo czy prosto.

**pacholek average** - funkcja bierze zbiór pachołków składowych, a następnie zwraca obliczony z nich nowy pachołek o uśrednionych współrzędnych.

**Działanie funkcji main**

Wczytujemy n i m, ustawiamy jakieś wartości dla minimów i maksimów (obecnie jest +100, -100, natomiast należy je dobrać tak, by były poza zakresem wartości które mogą być na wejściu)

W pętli wczytujemy wszystkie pachołki, ustawiamy, że początkowo każdy pachołek jest swoim własnym zbiorem o wielkości 1, i przy okazji znajdujemy minima i maksima.

Z ilorazu różnicy maksów i minów przez m uzyskujemy liczbę kolumn i wierszy, obie zwiększamy o 2 (to jest ta ramka wokół siatki).

Dla każdego pachołka obliczamy id komórki w której się znajduje.

Sortujemy po id - dla przykładowych danych to nie ma znaczenia, natomiast jeśli pachołki na wejściu mogą mieć id, które nie są w kolejności, to później nie bylibyśmy w stanie odnaleźć pozycji konkretnego pachołka na vectorze P, więc jest to tak dla bezpieczeństwa (jeśli byłaby gwarancja, że id na wejściu to kolejne liczby naturalne od 1 do n, to nie byłoby to potrzebne).

W głównej pętli przechodzimy po wszystkich pachołkach, określamy id jego komórki. t_index równa się g i nie jest konieczne, ale jest dla wygody, by się nie mylić z jednoliterkowymi zmiennymi.
Następuje pętla w pętli, obie wykonają się 3 razy, także łącznie 9 iteracji. temp_cell to id komórki sąsiadującej z tą aktualnie sprawdzaną. Jeśli w mapie nie istnieje klucz równy temp_cell, to znaczy, że nie ma w tej komórce żadnej zawartości (klucz nigdy nie został utworzony), więc nie musimy nic z nią robić
W przeciwnym wypadku coś musi być w komórce. Vector przypisany do temp_cell przechowuje wyłącznie id pachołków, dlatego (tutaj przydaje się pewność, że pachołki na vectorze P są posortowane po id) możemy z vectora P wyciągnąć konkretny pachołek, znając tylko jego id
Id pachołków zaczynają się od 1, numeracja na vectorze od 0, także odejmujemy 1.
Sprawdzamy, czy pachołki są bliżej niż m oraz czy nie są tożsame. Jeśli oba warunki są spełnione, to robimy unię, dodajemy do wspólnego zbioru.

next_id to będzie id scalonych pachołków, zaczynając od n + 1

bool was_message to kwestia wypisywania - na samym początku przed wypisywaniem nowych pachołków pojawia się tekst “New_ID, x, y, [Merged Cones]”, ale trochę głupio, jeśli nie mamy żadnego nowego pachołka, a to i tak się wypisze. Nie jest to koniecznie, ale wygląda ładniej.

Pętla wrzuca pachołki do vectora sets, dzięki temu jest szybki dostęp do danych pachołków znajdujących się w danym zbiorze.

Przechodzimy po wszystkich zbiorach, jeśli rozmiar jest mniejszy niż 2, to żadnych unii tam nie było, przechodzimy dalej.
Jeśli wiadomość początkowa nie została wyświetlona to wypisujemy ją. Tworzymy nowy pachołek z współrzędnymi obliczonymi przez funkcję average, i id równym next_id. Jest oznaczany jako aktywny (to ma znaczenie przy obliczaniu obwodu), ale id komórki nie ma znaczenia, dlatego jest zero.
Wypisujemy dane nowego pachołka oraz wypisujemy jego pachołki składowe (czyli wszystkie elementy znajdujące się w zbiorze i), które są oznaczane jako nieaktywne, więc nie będą brały udziału w tworzeniu otoczki.

Pierwsza część zadania jest wykonana.

# ZADANIE 1.2

## *Algorytm*

Wykorzystany algorytm to “Andrew's monotone chain convex hull algorithm”. Na wikibooks jest ładny gif, który przedstawia jego działanie. Zasadniczo bazuje on na fakcie, że iloczyn wektorowy pozwala określić, czy 3 punkty skręcają w lewo, prawo czy idą wprost. Skręt w prawo oznacza, że tworząc taką ścieżkę “musimy zawrócić”, a to nie jest optymalne, gdy szukamy najkrótszej otoczki.

## *Działanie*

Sortujemy po X, ustawiamy n na wartość next_id, bo jeśli dodaliśmy jakieś pachołki to chcemy też je badać.

Najpierw w pierwszej pętli budujemy dolną otoczkę, sprawdzamy, czy pachołek jest aktywny, jeśli nie to ignorujemy go.
Ponieważ hull.size() < 2 wrzucamy dwa pierwsze pachołki, czyli dwa skrajnie na lewo. Gdy już je mamy, to sprawdzamy warunek z iloczynem wektorowym, który jest mniejszy lub równy zero, gdy 3 punkty leżą na jednej prostej lub skręcają w prawo. Nie chcemy tego, więc gdy tak się dzieje, to usuwamy ostatni pachołek i wrzucamy nowy. Druga pętla buduje górną otoczkę, też sprawdzamy czy badany pachołek jest aktywny, a dodatkiem jest warunek z t, który gwarantuje, że nie usuniemy z vectora elementów dolnej otoczki. Usuwamy ostatni element, bo to duplikat pierwszego.

By obliczyć obwód takiej otoczki sumujemy wszystkie długości między dwoma kolejnymi pachołkami, używamy funkcji hypot, która oblicza przeciwprostokątną trójkąta prostokątnego o bokach równych różnicy w X i różnicy w Y. Finalnie wypisujemy obwód oraz indeksy pachołków składowych zgodnie z wymaganiami.

# EDGE CASE’Y I PROBLEMY

Jak pokazałam na początku, podejście zachłanne nie działa, także moja pierwsze rozwiązanie, mimo, że wydawało się sensowne, to niezbyt dobrze działało.

Pomysł na siatkę też nie był pierwszym, bo oczywiście istnienie brutalna opcja sprawdzania każdego punktu z każdym, co może działać (jeśli używamy Find-Union) ale złożoność obliczeniowa O(n^2) nie jest zbyt optymalna, a siatka pozwala ograniczyć się do złożoności liniowej przy porównywaniu punktów.

Siatka (gdy nie ma ramki z pustych komórek) wiąże się z ryzykiem wyjścia poza zakres, perspektywa tworzenia 8 warunków, by tego uniknąć, nie była zbyt zachęcająca, dlatego siatka zyskała takie zabezpieczenie.

W pierwotnym zachłannym rozwiązaniu używałam kolejki do sprawdzania punktów, co było sensowne, natomiast stan pachołków na kolejce niekoniecznie pokrywał się z faktycznym stanem, więc znalezienie tego problemu też zajęło chwilę.

## DOWÓD

Dowód, że sprawdzanie 9 komórek nie pominie żadnego punktu spełniającego wymagania d(P1, P2) < m

$$P1 = (x_1, y_1)$$
$$P2 = (x_2, y_2)$$

Komórki w siatce sa oznaczone przez pare (a, b), gdzie a to indeks kolumny, b to indeks wiersza:

$$a = \frac{x}{m}$$
$$b = \frac{y}{m}$$

Załóżmy, że odległość miedzy P1 i P2 jest mniejsza od m, wówczas

$$|x_1 - x_2| < m$$
$$a_1 = \frac{x_1}{m}$$
$$x_1 = a_1 m$$
$$a_2 = \frac{x_2}{m}$$
$$x_2 = a_2 m$$

Podstawiam obliczone wartosci do nierownosci

$$|a_1 m - a_2 m| < m$$
$$m|a_1 - a_2| < m$$
$$|a_1 - a_2| < 1$$

Analogicznie dzieje się z b, czyli w współrzędnych y. Z tego wynika, że jeśli dwa punkty mają między sobą odległość mniejszą niż m, to na pewno różnica między indeksami ich kolumn i wierszy nie jest większa niż 1, czyli sprawdzenie sąsiednich komórek wystarcza.
Dowody, że Find-Union i algorytm na otoczkę wypukłą działają są dobrze udokumentowane.